

Exercice 1 :**Implémentation de l'algorithme de Quine**

Le fichier **TP20.ml** contient une définition de la structure inductive **formules** que nous utiliserons. Il contient également des fonctions pour afficher des formules, et des fonctions permettant de calculer une valuation et d'afficher une table de vérité.

Dans cet exercice, nous considérons uniquement des formules sans $\leftrightarrow$.

1. Quelle est la complexité de **table_verite**?

Peut-on faire mieux si on veut seulement décrire (mais pas afficher) la table de vérité?

2. Écrire une fonction de signature **substitution** : **int** -> **formule** -> **formule** -> **formule** telle que **substitution** *i g f* calcule $f^{x_i \leftarrow g}$.

3. Dans cette question on cherche à mettre une formule sous NNF comme vu en cours. Contrairement à la preuve qu'on a faite, on fait l'élimination des implications en même temps que le reste.

- (a) Écrire une fonction **implique** : **formule** -> **formule** -> **formule** telle que **implique** *f1 f2* soit équivalente à $f_1 \rightarrow f_2$ par élimination du $\rightarrow$ à la racine.

- (b) Écrire une fonction **et** : **formule** -> **formule** -> **formule** telle que **et** *f1 f2* soit équivalente à $f_1 \wedge f_2$, et effectue la simplification si f_1 ou f_2 est dans $\{\perp; \top\}$.

Écrire de même **ou** : **formule** -> **formule** -> **formule**.

- (c) En déduire une fonction de signature **nnf** : **formule** -> **formule** qui met calcule une forme normale négative équivalente à son argument.

Attention aux multiples cas pour les formules commençant par une négation (c'est l'essentiel de ce qu'il reste à faire).

4. En utilisant **varmax**, **substitution** et **nnf**, on veut appliquer le principe du retour sur trace à la recherche d'un modèle d'une formule *f* :

- On pose *g* une NNF équivalente à *f* ;
- Si *g* est $\perp$ ou $\top$, alors on a une tautologie ou une antilogie et donc on sait si elle est satisfiable.
- Sinon, on identifie une variable *x* de *g*, et on applique l'algorithme à $g^{x \leftarrow \top}$. Si cette formule est satisfiable, on a fini. Sinon, on fait de même avec $g^{x \leftarrow \perp}$.

- (a) Écrire une fonction **quine_sat** : **formule** -> **bool** qui détermine simplement si une formule est satisfiable.

Vérifier que $(X_1 \rightarrow (\neg X_2)) \wedge (X_2 \rightarrow X_1)$ est satisfiable, et que $\neg((X_1 \rightarrow (\neg X_2)) \wedge (X_2 \rightarrow X_1)) \wedge \neg X_2$ ne l'est pas.

- (b) Écrire une fonction **quine_modele** : **formule** -> **bool list option** qui renvoie **Some** de la liste des valeurs de vérité des variables dans un modèle de *f* si un modèle existe, et **None** sinon.

- (c) Écrire une fonction **quine_affiche_modeles** : **formule** -> **unit** qui affiche tous les modèles d'une formule, en ne précisant pas la valeur des variables qui ne sont pas pertinentes. Chaque ligne correspond à un (ou des) modèles différents, et aucun modèle ne peut correspondre à plusieurs lignes.

Par exemple, avec T_f ci-dessous :

x_1	x_2	x_3	<i>f</i>
V	V	V	V
V	V	F	V
V	F	V	V
V	F	F	F
F	V	V	V
F	V	F	F
F	F	V	V
F	F	F	V

On pourra se contenter d'afficher :

```

x1 = V    x2 = V
x1 = V    x2 = F    x3 = V
x1 = F    x2 = V    x3 = V
x1 = F    x2 = F    x3 = V
x1 = F    x2 = F    x3 = F

```

Mais on pourrait aussi afficher par exemple (avec une stratégie d'instanciation des variables différentes) :

```

x3 = V
x1 = V    x2 = V    x3 = F
x1 = F    x2 = F    x3 = F

```

En pratique, chaque ligne affichée correspondra à une situation où, étant donné ces substitutions, l'algorithme de Quine a trouvé une tautologie (ce qui n'est pas forcément immédiatement le cas même si f est une tautologie).

5. (*Chez vous, question ouverte*) On se donne une CNF sous la forme d'une liste d'éléments `int list` qui représentent ses clauses disjonctives, avec le signe de chaque entier (toujours non nul) i indiquant si le littéral représenté est X_i ou $\neg X_{-i}$.

Écrire une fonction `quine_cnf_sat : int list list -> bool` qui détermine si cette CNF est satisfiable.

Vous pouvez chercher à améliorer le choix des variablesinstanciées en utilisant des parcours de liste intelligemment.

Exercice 2 :

Réécriture d'un problème sous forme de CNF

On s'intéresse au problème des n reines sur un échiquier de taille $n \times n$.

On pose $x_{ni+j+1} \in \mathcal{V}$ la variable propositionnelle qui signifie "il y a une reine à la case de ligne i et de colonne j ", pour $0 \leq i, j \leq n-1$.

On remarque que l'indice des variables propositionnelles va de 1 à n^2 , ce qui facilite le fait de représenter les littéraux comme des entiers relatifs.

1. Exprimer comme une formule CNF la proposition "il y a au plus une reine sur la ligne i ". Quelle est la taille de cette formule, en fonction de n ?
2. En déduire la taille d'une formule CNF exprimant la proposition "il y a au plus une reine par ligne et une reine par colonne".
3. (a) Exprimer comme une formule CNF la proposition "Il y a au plus une reine sur la diagonale allant de $(0; i)$ à $(i; 0)$ ".
(b) Donner, en fonction de n et de $i \in \llbracket 0; n-1 \rrbracket$, la taille des formules CNF exprimant respectivement les propositions "Il y a au plus une reine sur la diagonale allant de $(0; i)$ à $(i; 0)$ " et "il y a au plus une reine sur la diagonale allant de $(0; i)$ à $(n-i-1; n-1)$ ".
4. En déduire, en fonction de n , la taille d'une formule CNF exprimant la proposition "Il y a au plus une reine par colonne, par ligne, et par diagonale".
5. (a) Que manque-t-il pour caractériser entièrement les solutions du problème des n reines ?
(b) Si on exprime naïvement la condition manquante, quelle est la taille (en fonction de n) de la CNF à ajouter ?
(c) En utilisant ce qu'on sait sur la répartition des reines dans une solution qui respecte les contraintes précédentes, comment peut-on exprimer cette même condition avec une CNF beaucoup plus petite ?
(d) En déduire la taille d'une formule CNF dont les modèles correspondent exactement aux solutions du problème des n reines.
6. Écrire le problème des 3 reines sous forme de CNF comme ci-dessus. En appliquant à la main l'algorithme de Quine, montrer qu'il n'a pas de solution.

On pensera à simplifier en premier lieu les clauses disjonctives réduites à un littéral. Il est évidemment possible de faire en une seule fois plusieurs substitutions nécessaires (i.e. on substitue $\perp$ à x et $\top$ à y car $\neg x$ et y sont des clauses disjonctives de la CNF, et toute autre substitution pour ces variables donne trivialement une antilogie).

7. (*Chez vous*) Implémentez une fonction `cnf_reines : int -> formule` qui calcule une CNF du problème des n reines.

Observer que résoudre le problème des n reines avec cette fonction et l'algorithme de Quine est possible mais en pratique plus lent que le résoudre avec un algorithme de backtracking dédié au problème.